Report

For this assignment, I selected the f11 snapshot of the dataset with users. It contains all 5 required CSV files: friends.csv, messages.csv, events.csv, client_first_purchase_date.csv, campaigns.csv.

I. Exploratory Data Analysis

To begin, I decided to conduct a small Exploratory Data Analysis (EDA) of the data for which we will build data models.

1. Data Loading and General Information

- The data was extracted from the zip archive and loaded using the Pandas library.
- For each dataset, information about the number of rows and columns was displayed, and the info() method was called, showing data types and the number of non-null values. All files have different sizes and numbers of features:
    - campaigns: 1907 rows, 19 columns.
    - first_purchase: 176,162 rows, 4 columns.
    - events: 1,290,282 rows, 9 columns.
    - friends: 1,974,239 rows, 2 columns.
    - messages: 3,009,333 rows, 34 columns.
- The campaigns file turned out to be significantly smaller than the others in terms of number of rows.

2. Splitting Columns into Categorical and Numerical

For each dataframe, columns of type object (categorical) and numerical were identified:

- In campaigns, 14 are categorical, 5 are numerical.
- In first_purchase, only first_purchase_date is categorical, 3 are numerical.
- In events, 5 are categorical (event_time, event_type, category_code, brand, user_session), 4 are numerical.
- In friends, all columns are numerical.
- In messages, 28 are categorical, 6 are numerical.

- Specific types (e.g., date and time, boolean) will require subsequent explicit type conversion for correct storage.

3. Checking for Duplicates

The number of completely duplicated rows was counted:

- In campaigns, first_purchase, events, messages, duplicates are absent. The count is 0.
- In friends, one duplicated row was found. Further analysis showed it was the pair (friend1=546810522, friend2=558794622), occurring twice.
- Since in this context friendship is bidirectional, (friend1)-[is_friend]-(friend2) is equivalent to (friend2)-[is_friend]-(friend1). Therefore, for friends, I also performed a permutation of friend IDs and identified that after permutation, there are 7 duplicate friend pairs.

4. Detailed Analysis of events

- Time interval of event_time: from 2019-10-01 00:00:00 to 2019-12-31 23:55:44.
- Distribution of event types: views -- 1,216,009, cart additions -- 51,188, purchases -- 23,085.
- Unique products: 90,320, unique users: 65,444, unique sessions: 282,556.
- Missing values: in the category_code column, 330,172 values are missing (about 25.6%), in brand -- 175,339 (about 13.6%). The remaining columns are completely filled.
- Missing value overlap: in 86,848 rows (6.73%), both brand and category_code are missing simultaneously.
- The relationship between category_id and category_code was investigated: for each category_id, there is only one unique category_code value (excluding empty values), which generally confirms that category_code is a textual representation of the category identifier.
- An attempt to fill missing values in category_code based on grouping by category_id reduced the number of missing values from 330,172 to 146,590, however about 11.4% of records still lack a category code.
- Checking the correspondence between user_session and user_id: all sessions belong to only one user (no sessions with different user_ids), there are no missing values in these fields.

5. Analysis of first_purchase

- No duplicates.
- Checking the formation of client_id: for all records, the client_id value matches the concatenation of the string prefix "151591562", user_id, and user_device_id.
- No missing values in any column.
- Grouping by client_id shows that each client corresponds to exactly one user_id and one user_device_id (unique correspondence).

6. Analysis of campaigns

- Distribution of campaign types: bulk -- 1830, transactional -- 50, trigger -- 27.
- Channels: mobile_push -- 1396, email -- 483, multichannel -- 27, sms -- 1.
- Time frames: campaigns ran from 2021-04-30 to 2023-04-21.
- No duplicates.
- Missing values: the largest number of missing values in columns ab_test (1895 missing, i.e., almost all), hour_limit (1876), is_test (1880), position (1883), as well as partially in topic, started_at, finished_at, total_count, warmup_mode, subject_length, and columns related to the email subject.
- Checking the uniqueness of the combination of id and campaign_type: all pairs are unique, indicating a possible composite key.
- Analysis by channel: only for the "multichannel" channel are the position and is_test fields filled, with is_test taking False and True values; for other channels, these fields are completely absent.
- For the warmup_mode column: missing values are present in campaigns of type 'trigger' and 'transactional', while for type 'bulk' there are no missing values.
- Date comparison: there is not a single row where started_at is greater than finished_at.

7. Analysis of friends

- One duplicate row was found (friend1=546810522, friend2=558794622), occurring twice.
- Since in this context friendship is bidirectional, (friend1)-[is_friend]-(friend2) is equivalent to (friend2)-[is_friend]-(friend1). Therefore, for friends, I additionally performed a permutation of friend IDs and identified that after permutation, there are 7 duplicate friend pairs.
- No missing values.
- Check for "self-friendship" (friend1 = friend2) revealed no such cases.

- Distribution of degrees (number of friends per user): average ~6, minimum 1, maximum 21, median 6, standard deviation about 2.43. Total unique users in the graph: 656,456.
- A histogram of the distribution of the number of friends was plotted.

8. Analysis of messages

- Time ranges:
    - date (sending date) from 2021-04-30 to 2021-06-14.
    - sent_at from 2021-04-30 00:05:45 to 2021-06-14 23:58:24.
    - opened_first_time_at from 2021-04-30 00:08:55 to 2022-01-20 14:10:46.
    - opened_last_time_at from 2021-04-30 00:08:55 to 2022-01-20 14:10:46.
    - clicked_first_time_at from 2021-04-30 00:55:57 to 2021-11-19 13:40:53.
    - clicked_last_time_at from 2021-04-30 00:55:57 to 2021-11-19 13:40:53.
- Unique clients in messages: 1,007,461, unique campaigns: 66.
- Distribution of message types: bulk -- 2,121,331, trigger -- 645,924, transactional -- 242,078.
- Channels: email -- 1,742,210, mobile_push -- 1,267,042, web_push -- 81.
- Checking timestamp consistency: no rows found where opened_first_time_at > opened_last_time_at.
- No duplicates.
- Missing values:
    - The category column is completely missing (3,009,333 missing, 100%).
    - Very high percentage of missing values in columns related to actions: opened_first_time_at and opened_last_time_at -- about 83.8%, clicked_first_time_at and clicked_last_time_at -- about 97.6%, unsubscribed_at -- 97%, hard_bounced_at -- 99.7%, etc.
    - platform is missing in 92.6% of records, email_provider -- in 42.1%.
- The stream column contains only one unique value.
- Dependency check: for each client_id, there is only one user_id and one user_device_id (as in first_purchase).
- Comparison of the set of clients from messages and first_purchase: 901,340 clients are present in messages but absent in first_purchase; 70,041 clients from first_purchase are absent in messages. This indicates a significant discrepancy between clients who received messages and those who made their first purchase. Probably, for completeness of client information, when modeling, it will be necessary to supplement first_purchase with the missing ids.

General Conclusions

- All datasets contain no duplicates, except for friends.
- In each dataset, there are missing values, especially pronounced in messages and some columns of campaigns.
- Key fields (identifiers) are consistent within datasets: client_id uniquely determines user_id and user_device_id in first_purchase and messages. This may be useful for normalization.
- For each category_id, there is only one unique category_code value (excluding empty values), which generally confirms that category_code is a textual representation of the category identifier. This may be useful for normalization.
- A significant proportion of clients in messages do not have a record of their first purchase, which may indicate that messages were sent before a purchase was made or that clients never purchased at all.

II. Data Cleaning

1. General Characteristics

The script performs preprocessing of five source CSV files to bring them into a unified, consistent format suitable for loading into databases. Main tasks: removing duplicates, correcting data types, filling some missing values, and normalizing relationships.

2. Cleaning events.csv

- Duplicate rows removed -- ensures uniqueness of event records.
- The event_time column converted to datetime format with UTC timezone; erroneous values replaced with NaT (not a time). This ensures correct chronology.
- Missing values in category_code filled with the first encountered value for the same category_id. This is acceptable since category_code is a textual representation of the category identifier. As a result, the number of missing values decreased, and data integrity improved.
- price converted to Float32 type.
- user_id converted to string -- unification with other files.

Result: events are fully typed, technical duplicates eliminated, some missing category codes restored.

3. Cleaning campaigns.csv

- Duplicates removed -- each campaign is represented by a unique row.
- Boolean columns (ab_test, warmup_mode, etc.) transformed: empty values became pd.NA, strings 'true' interpreted as True, everything else as False.
- Timestamps started_at, finished_at converted to datetime; invalid values become NaT.
- Numerical fields (total_count, hour_limit, subject_length, position) converted to numbers; conversion errors give NaN. For integer fields, Int64 type (allowing NULL) was used.

Result: all campaign attributes have correct types, boolean values standardized.

4. Cleaning messages.csv

- Duplicates removed -- each message is unique.
- Date and time (date, sent_at) converted to datetime; invalid -- NaT.
- Eight boolean flags (is_opened, is_clicked, etc.) explicitly converted to bool type. This ensures they contain only True/False, even if the source data was in a different format.
- Event timestamps (opening, click, unsubscribe, etc.) also converted to datetime.
- All text identifiers (client_id, user_id, message_id, message_type, channel, category, platform, email_provider, stream) converted to strings.

Result: messages are fully typed, and keys for linking with other files have the same string format.

5. Cleaning client_first_purchase_date.csv

- Duplicates removed.
- Critical addition: the source file might have lacked records for some client_ids present in messages.csv. The script finds such client_ids and adds them to the first purchase table, using data from messages (client_id, user_device_id, user_id). This restores data completeness -- now every client who received a message will have a first purchase record (even if with an empty date, if no purchase yet occurred).
- Type conversion: client_id, user_id, user_device_id became strings, first_purchase_date became datetime.

Result: the first purchase table now contains all client_ids appearing in messages, ensuring referential integrity in future joins.

6. Cleaning friends.csv

- Direction normalization: pairs friend1 and friend2 are sorted so that friend1 < friend2 always. This eliminates duplication due to different ordering (e.g., (A,B) and (B,A) become one record).
- Duplicates removed after sorting.
- Both identifiers converted to string.

Result: the friendship graph has become uniformly directed, each relationship is stored once, simplifying queries and preventing double counting.

7. General Conclusions on Data Quality and Cleaning Optimality

- Eliminating duplicates in all files guarantees that each entity is represented once, which is critical for primary keys.
- Imputation was not performed, as it could create unnecessary bias. Except for category_code, which is a textual analogue of category_id, which was available.
- Type conversion (datetime, boolean, numbers, strings) ensures compatibility with target DBMS (PostgreSQL, MongoDB, Neo4j) and allows correct comparisons, sorting, and aggregations.
- Filling missing values in category_code and adding missing records to first_purchase increases data completeness without information loss.
- Normalization of friendship ties eliminates ambiguity and prepares data for graph queries.
- All transformations are performed considering possible errors (parameter errors='coerce'), so the script is resilient to "dirty" data.

Thus, the cleaning is optimal because it:

- minimizes data loss,
- brings data to a common standard,
- restores missing connections,
- and prepares the sets for efficient loading into various databases without needing reprocessing.

III. Data Models

III.I. PostgreSQL

General Description of the Data Model

The model represents a normalized relational schema consisting of 8 tables that fully reflect the structure of the five source CSV files and the relationships between them. The main goal of the schema is to ensure data integrity, minimize redundancy, and create a foundation for effective analytics as described in the assignment.

Here's how the CSV files are distributed across tables:

- users -- a directory of all unique users (user_id) collected from all sources.
- categories -- a directory of product categories from events.csv.
- session -- a directory of user sessions from events.csv.
- event -- stores all rows from events.csv, linking them to sessions and categories.
- first_purchase -- stores all data from client_first_purchase_date.csv.
- campaign -- stores all data from campaigns.csv.
- friends -- stores undirected friendship ties from friends.csv.
- messages -- stores all data from messages.csv.

Data Model Schema

Relationships between tables are built using foreign keys, which ensures referential integrity.

- users (user_id) -- is the root table. It is referenced by:
    - session (user_id)
    - first_purchase (user_id)
    - friends (friend1, friend2)
- categories (category_id) -- directory. It is referenced by event (category_id).
- session (user_session) -- contains unique sessions. It is referenced by event (user_session).
- campaign (id) -- campaign directory. It is referenced by messages (campaign_id).
- first_purchase (client_id) -- contains unique client identifiers. It is referenced by messages (client_id).
- friends (friend1, friend2) -- a many-to-many relationship table between users, implemented as an undirected graph with a check friend1 < friend2.
- event and messages -- fact tables, accumulating all events and messages.

Optimality

1. Compliance with business requirements (from the assignment):
    - Volume and Variety: The model separates heterogeneous data (behavioral events, messages, social graph, directories) into different tables. This allows independent management of the growth of each data type.
    - Value: The data in the model is normalized (split into different tables), which optimizes storage. A separate Users table is allocated, and indexes are added to speed up queries.
2. Data Integrity and Consistency:
    - The use of PRIMARY KEYS guarantees the uniqueness of each record in directories.
    - FOREIGN KEYS (REFERENCES) ensure that each record in fact tables references an existing entity. This is critically important for data quality when scaling.
3. Elimination of Redundancy (Normalization):
    - User information is stored only in the users table, category information only in categories, etc. This saves space and eliminates update anomalies (e.g., to correct a category code, you only need to change it in one place).

Optimization Methods

1. Indexing Foreign Keys: Immediately after table creation, indexes are added to all columns that are foreign keys:
    - idx_session_user_id on session(user_id)
    - idx_event_category_id on event(category_id)
    - idx_event_user_session on event(user_session)
    - idx_first_purchase_user_id on first_purchase(user_id)
    - idx_friends_friend2 on friends(friend2) --- this is critically important for bidirectional search on the friend graph, as the primary key only covers search by friend1.
    - idx_messages_campaign_id and idx_messages_client_id on messages(campaign_id, client_id)
    - These indexes speed up JOIN operations between tables.
2. Integrity Check at the DB Level: The CHECK constraint (friend1 < friend2) in the friends table guarantees unique storage of undirected relationships and prevents duplication ((user1, user2) and (user2, user1)).

3. Choice of Data Types: Using TEXT for identifiers, BIGINT for large numbers, NUMERIC for prices, and TIMESTAMP for time provides sufficient precision and range for all scenarios.

III.II. MongoDB

General Description of the Data Model

The model represents a denormalized document schema consisting of 4 collections that directly correspond to the five source CSV files. The main principle is that data is organized not into linked tables, but into self-contained documents, optimized for potential use cases.

Here's how the CSV files are distributed across collections:

- campaigns --- stores all data from campaigns.csv. The campaign_id field is represented as a nested document {id, campaign_type}, which solves the problem of non-unique campaign identifier without type.
- users --- the central collection for users. It embeds:
    - A list of friends (friends) from friends.csv.
    - An array of first purchases (first_purchases) from client_first_purchase_date.csv for each client_id associated with the user.
- messages --- stores all data from messages.csv. It contains denormalized information about the campaign (campaign), allowing reading a message along with its campaign context without a separate query.
- events --- stores all data from events.csv. It contains denormalized information about the product (product) directly in the event document.

Data Model Schema

The model uses embedding for one-to-many and many-to-many relationships where efficient, and references for relationships with large separate entities.

- campaigns --- independent collection. The document contains all campaign fields, with subject properties grouped into a subdocument subject. campaign_id is a composite identifier.
- users --- collection into which are embedded:
    - friends: an array of friend identifiers (user_id).

    - first_purchases: an array of documents with first purchase details for each client_id of the user.
- messages --- collection that references campaigns via campaign_id, but also embeds denormalized copies of campaign fields (campaign.type, campaign.channel, campaign.topic) for fast access. The document contains all message fields, grouped into subdocuments timing, flags, metadata.
- events --- collection that embeds complete product information (product) directly in the event document. It contains all fields from events.csv, with user_session renamed to session_id.

Optimality

1. Read Performance:
    - Denormalization in the messages and events collections ensures that to perform certain analytical queries, MongoDB only needs to read one collection (messages or events) or perform a simple join by user_id, which is significantly faster than JOINing multiple tables.
    - Embedding the friends list in the users document means that to obtain a user's social graph, you don't need to access a separate collection --- all data is already in one document.
2. Scalability:
    - MongoDB scales horizontally excellently. In this model, the events collection is the fastest-growing, and it can be easily sharded by user_id. The messages collection can also be sharded by user_id or campaign_id. The users and campaigns collections will grow more slowly and can be stored on separate nodes.

Optimization Methods

1. Data Denormalization:
    - In the messages collection, copies of fields from campaigns (campaign.type, campaign.channel, campaign.topic) are stored. This eliminates the need to perform a separate query to the campaigns collection when analyzing messages.
    - In the events collection, a complete nested product document is stored. This eliminates the need to create a separate collection for products and speeds up all product-related queries.
    - In the users collection, friends lists and first purchase histories are stored.
2. Strategic Indexing: The code creates many indexes to cover main queries:

    - Unique indexes: on users.user_id, campaigns.campaign_id (composite) and messages.id to ensure integrity and fast ID lookup.
    - Indexes for filtering and sorting: on messages.user_id, messages.campaign_id, messages.sent_at, events.user_id, events.session_id, events.event_time, events.event_type. This is critical for time series and filtering.
    - Composite indexes: e.g., [('user_id', ASCENDING), ('sent_at', DESCENDING)] in the messages collection for efficiently retrieving a user's latest messages.
    - Text index (TEXT) in the events collection on the fields product.category_code and product.brand.
3. Using Nested Documents to Group Related Fields: In campaigns, subject fields are grouped into a subject subdocument. In messages, all timestamps are grouped into timing, and flags into flags. This improves readability and data organization.

III.III. Neo4j

General Description of the Data Model

The model represents a graph schema consisting of 3 node types and 4 relationship types that directly correspond to the five source CSV files. The main principle is that data is organized as connected entities (nodes) and interactions between them (edges).

Here's how the CSV files are distributed in the graph:

- User nodes --- contain only the user identifier (user_id), collected from all sources (files first_purchase, events, messages, friends).
- Campaign nodes --- contain all fields from campaigns.csv. They are identified by the pair (campaign_id, campaign_type), which solves the problem of non-unique campaign identifier without type.
- Category nodes --- contain category_id and category_code from events.csv.
- FRIEND relationships --- connect User nodes to each other, forming a bidirectional social graph based on friends.csv.
- VIEWED / PURCHASED relationships --- connect User nodes to Category nodes and contain in their properties the complete event information from events.csv: time, session identifier, as well as product details (product_id, brand, price).

- RECEIVED relationships --- connect User nodes to Campaign nodes and contain in their properties all fields from messages.csv. This is a key decision, turning messages into a relationship between a user and a campaign.
- FIRST_PURCHASE relationships --- are self-relationships (loops) on the User node and store in their properties information about the client's first purchase from client_first_purchase_date.csv (client_id, first_purchase_date, user_device_id).

Data Model Schema

The model uses nodes to represent main entities and edges to represent all interactions and relationships between them.

- Nodes:
  - User (user_id)
  - Campaign (campaign_id, campaign_type, and all other fields from CSV)
  - Category (category_id, category_code)
- Relationships:
  - (:User)-[:FRIEND]-(:User) --- bidirectional friendship.
  - (:User)-[:VIEWED {event_time, session_id, product_id, brand, price}]->(:Category) --- view event.
  - (:User)-[:PURCHASED {event_time, session_id, product_id, brand, price}]->(:Category) --- purchase event.
  - (:User)-[:RECEIVED {all fields from messages.csv}]->(:Campaign) --- receiving a message from a campaign.
  - (:User)-[:FIRST_PURCHASE {client_id, first_purchase_date, user_device_id}]->(:User) --- self-loop, marking the fact of a first purchase.

Optimality

1. Natural Representation of Connected Data: The main business value, according to the problem statement, lies in analyzing social networks for targeted marketing. In the graph model, friendship ties (FRIEND) help efficiently and intuitively navigate to find a user's friends, their interests, and behavior.
2. Modeling Behavior as Relationships: View and purchase events (VIEWED, PURCHASED) are modeled not as separate nodes, but as relationships between a user and a category. This allows storing all event attributes (time,

product, brand, price) directly on the relationship and easily answering questions like "what did my friend's friends buy?" by simple graph traversal.
3. Efficient Message Analysis: Modeling message receipt as a RECEIVED relationship between a user and a campaign, rather than as a separate node, is a key graph pattern. This turns communication history into a set of relationships that can be analyzed together with other data.
4. Traversal Performance: Unlike relational databases, where complex JOINs of multiple tables can be slow, in a graph database, traversal performance depends only on the size of the traversed part of the graph, not on the total data volume. This is critical for scenarios involving social ties ("friends of friends").

Optimization Methods

1. Using Relationships Instead of Nodes for Messages and Events: Instead of creating nodes for each message or event, they become edges. This reduces the number of nodes and makes graph traversal more direct and faster.
2. Indexing Key Properties:
    - Indexes created for fast node lookup by identifiers: :User(user_id), :Campaign(campaign_id, campaign_type), :Category(category_id).
    - Indexes created on relationship properties that will be frequently used in filtering and sorting: :RECEIVED(sent_at), :RECEIVED(is_purchased), :VIEWED(event_time), :PURCHASED(event_time). This allows efficiently finding relationships for a certain period or with certain flags.
3. Bidirectional FRIEND Relationships: Creating two directed relationships ((u1)-[:FRIEND]->(u2) and (u2)-[:FRIEND]->(u1)) doubles the volume, but guarantees that graph traversal in any direction will be equally fast and simple, without needing to check both directions in the query.

General Description
The data model in ArangoDB is multi-model, combining document and graph capabilities. It consists of 6 document collections and 1 edge collection that directly correspond to the five source CSV files. Social connections between users are separated into a dedicated graph for efficient traversal, while the bulk of the data is stored as self-contained documents.

Data Model Schema

- Document collections:
    - users (_key: user_id, friends field --- array of friend identifiers).
    - categories (_key: category_id, category_code).
    - campaign (_key: id, all fields from campaigns.csv).

    - first_purchase (_key: client_id, fields user_id, user_device_id, first_purchase_date).
    - events (auto-_key, fields: user_id, event_time, event_type, session_id, product_id, category_id, brand, price).
    - messages (_key: id, all fields from messages.csv).
- Edge collection:
    - friends --- stores edges between users (_from, _to), representing undirected relationships in a directed form (with sorting friend1 < friend2).

Optimality

1. Multi-model Approach: The majority of the data (campaigns, events, messages) is stored in documents, which is convenient for filtering and analytics. The social graph is separated into dedicated edges, which allows efficient graph traversals (e.g., finding friends of friends).
2. Query Versatility: The AQL language allows combining document queries with graph traversals, which is suitable for tasks requiring analysis of both the data itself and the connections between them.
3. Use of System Keys (_key) for direct access by identifier speeds up read and write operations.

Optimization Methods

- Indexing: hash indexes created on user_id fields (in events), campaign_id and client_id (in messages) to speed up filtering and joins.
- Denormalization: the friends array in the users document duplicates graph relationships for fast access in simple queries.

Comparative Table of Data Models

Model	Main Advantage	Main Disadvantage	Optimizations Used
PostgreSQL (relational)	Data integrity -- strict schema, foreign keys and	Scaling complexity -- horizontal scaling is difficult,	Indexes on all foreign keys (to speed up JOINs).

	constraints guarantee consistency.	JOINs become slow as data grows. Less flexible schema.	Normalization of data.
MongoDB (document)	Schema flexibility -- easily adapts to data changes. High read speed thanks to denormalization and embedding of related documents.	Data duplication -- denormalization leads to redundancy and complicates updates.	Indexes (including composite and full-text). Denormalization.
Neo4j (graph)	Natural representation of relationships -- ideal for social networks and recommendations.	Not optimized for analytical queries over large volumes of event data. Complexity of performing aggregations compared to SQL.	Indexes. Bidirectional FRIEND relationships for fast traversal in both directions. Storing event attributes directly on edges.

ArangoDB (multi-model)	Versatility -- combines document storage and graph traversals. Allows choosing the optimal method for each part of the data (events -- in documents, friendships -- in graph).	Design complexity -- requires deciding where to use graph and where to use documents.	Hash indexes on frequently queried fields (user_id, campaign_id, client_id). Use of _key for direct access by identifier. Denormalization of friend list in user document (for fast simple queries).

Query Analysis

To test the performance of each of the first 3 data models, I implemented similar queries written separately for each data model. I ran each query 5 times and recorded the query processing time, then calculated the average. Unfortunately, a failure occurred with Neo4j, so I was only able to measure time on PostgreSQL and MongoDB.

```
================================================================================
FINAL STATISTICS
================================================================================
Query     | Database   | Run1      | Run2      | Run3      | Run 4     | Run5      | Average   | StdDev
--------------------------------------------------------------------------------------------------
q1_psql   | PostgreSQL | 202.290   | 205.331   | 199.983   | 209.657   | 206.320   | 204.716   | 3.336
q1_mongo  | MongoDB    | 2171.618  | 2133.531  | 2179.915  | 2201.235  | 2198.161  | 2176.892  | 24.34
q2_psql   | PostgreSQL | 3.491     | 1.472     | 1.203     | 1.258     | 1.180     | 1.721     | 0.996
q2_mongo  | MongoDB    | 10.055    | 7.746     | 6.859     | 6.862     | 7.161     | 7.737     | 1.345
q3_psql   | PostgreSQL | 9.992     | 9.458     | 8.886     | 8.984     | 9.027     | 9.269     | 0.460
q3_mongo  | MongoDB    | 459.401   | 453.759   | 462.792   | 467.401   | 474.632   | 463.597   | 7.926
```

The final execution time on PostgreSQL turned out to be more efficient compared to MongoDB.